Lifecycle Re-entry Control Module (LRECM)

Lifecycle Re-entry Control Module (LRECM)

OOF® Origin Open Foundation®

Independent Methodological Authority

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Parent Standard: Software Change Management Standard (SCMS)

Category: Governance & Enforcement

Subcategory: Software Change Management Governance

Type: Core Module

Governed Space: Software Change Lifecycle Re-entry Governance

Module ID: SLGA-SCMS-LRECM-04

Origin ID: OOF-SLGA-SCMS-LRECM

Version: 1.0

Status: Canonical Open

Effective Date: 14.10.2026

Compatibility: Methodology OS — Modular Compatibility

GIE-Compatible: Yes

AI-Readable: Yes

Authority: OOF® — OriginOpen® Foundation

Protection: Protected Methodological Work

Canonical Language: English

Canonical Definition

Lifecycle Re-entry Control Module (LRECM) is the Core Module within Software Change Management Standard (SCMS) that governs how proposed, approved, or materially evolving software changes are directed into the applicable software lifecycle governance mechanisms according to their assessed consequences, affected software objects, existing evidence, operational conditions, and required verification obligations.

LRECM establishes lifecycle re-entry as a controlled, evidence-supported

relationship between an identifiable software change and the specific lifecycle governance activities that must be performed, repeated, extended, or verified because of that change.

The module distinguishes the need to revisit a lifecycle governance

mechanism from the need to repeat every activity previously performed within it, the existence of historical verification evidence from its continued applicability, and the authorization of a change from the satisfaction of its lifecycle obligations.

It requires each material lifecycle re-entry decision to preserve its

basis, affected governance mechanisms, entry conditions, required activities, evidence reuse decisions, completion criteria, applicable authority, and unresolved obligations.

LRECM recognizes that software changes may originate at any lifecycle stage and that their consequences may require movement backward, forward, or across the established software lifecycle governance structure.

It applies to human-directed modifications, automated development

processes, dependency changes, security remediation, production corrections, AI-generated software changes, autonomous agents, and software changes affecting embedded, industrial, or robotic systems.

Its principal governance output is the Software Lifecycle Re-entry Profile .

Operational Role

LRECM establishes the governance mechanism through which software changes are connected to the lifecycle activities affected by their consequences.

It governs:

• Lifecycle re-entry identification

• Change-to-lifecycle mapping

• Re-entry scope

• Affected Parent Standards

• Applicable Core Modules

• Re-entry triggers

• Lifecycle entry conditions

• Required reassessment

• Required verification

• Historical evidence applicability

• Evidence reuse

• Reverification requirements

• Lifecycle activity sequencing

• Cross-standard dependencies

• Applicable authority references

• Re-entry exceptions

• Emergency lifecycle pathways

• Automated lifecycle re-entry

• Autonomous agent boundaries

• Re-entry monitoring

• Re-entry deviations

• Completion evidence

• Residual lifecycle obligations

• Lifecycle re-entry continuity

• Handoff to change closure

LRECM does not replace the specialized governance requirements of the lifecycle standards into which a change is directed.

Operational Question

How can a software change be directed back into precisely those lifecycle governance mechanisms materially affected by its consequences, while preserving applicable evidence, avoiding unjustified repetition, preventing necessary verification from being bypassed, and ensuring that every required lifecycle obligation remains identifiable and verifiably satisfied?

Position in Architecture Governance Flow

SLGA™ Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-cutting: Software Lifecycle Control

SCMS Core Module Flow

Change Request Management Module (CRMM) → Change Impact Assessment Module

(CIAM) → Change Coordination Control Module (CHCM) → Lifecycle Re-entry Control Module (LRECM) → Change Closure Management Module (CCLM)

LRECM is the fourth Core Module within SCMS.

CRMM establishes the change request.

CIAM identifies the consequences and affected governance mechanisms.

CHCM establishes material coordination relationships, prerequisites, and

sequencing conditions.

LRECM determines the required lifecycle governance pathway and controls the

traceability of the change through the applicable lifecycle mechanisms.

CCLM subsequently evaluates whether the resulting change and its

obligations have been appropriately resolved.

Lifecycle re-entry may be reassessed repeatedly when new evidence, changed scope, or operational deviations alter the applicable governance requirements.

• Governed Space

Software Change Lifecycle Re-entry Governance

This governed space concerns the controlled return, continuation, or

transfer of software changes into lifecycle governance mechanisms affected by the proposed or implemented modification.

It begins when an identifiable change requires determination of the lifecycle activities necessary to establish a legitimate resulting software state.

It includes:

Lifecycle Re-entry Determination

Affected Standard Identification

Required Activity Selection

Evidence Applicability Assessment

Lifecycle Sequencing

Entry Condition Verification

Re-entry Execution Traceability

Completion Verification

Residual Obligation Preservation

It continues until the applicable lifecycle obligations have been evidenced

as satisfied, legitimately deferred or excepted under competent governance, or transferred as unresolved conditions.

Governed Object

The primary Governed Object is:

A Software Lifecycle Re-entry Case representing the evidence-supported

relationship between an identifiable software change, its assessed consequences, the affected software lifecycle governance mechanisms, applicable historical evidence, required activities, entry conditions, completion requirements, and resulting lifecycle governance status.

The governed object must preserve, where applicable:

Lifecycle Re-entry Case ID

Change Request ID

Change Impact Assessment Reference

Change Coordination Reference

Affected Software Objects

Current Lifecycle State

Required Lifecycle Mechanisms

Re-entry Scope

Entry Conditions

Required Activities

Evidence Reuse Decisions

Required Reverification

Authority References

Completion Criteria

Execution Evidence

Residual Obligations

Re-entry Status

Re-entry Version

Scope

LRECM governs:

Lifecycle Re-entry Recognition

Re-entry Case Identification

Lifecycle Impact Mapping

Affected Parent Standard Identification

Affected Core Module Identification

Re-entry Necessity Assessment

Re-entry Scope Definition

Historical Evidence Review

Evidence Reuse Determination

Reverification Requirements

Lifecycle Entry Conditions

Lifecycle Sequencing

Cross-Standard Dependencies

Re-entry Authority References

Re-entry Exceptions

Emergency Re-entry

Automated Re-entry

Agent-Initiated Re-entry

Lifecycle Completion Evidence

Re-entry Reassessment

Residual Obligation Control

Closure Handoff

LRECM does not prescribe a specific development process, workflow automation platform, release pipeline, or organizational approval structure.

Foundational Principle

A software change must re-enter every lifecycle governance mechanism materially affected by the change, but it must not be forced through unrelated mechanisms without a justified governance need.

Lifecycle governance must be proportional to the actual change consequences.

A documentation correction may require limited governance activity.

A security-sensitive dependency replacement may require dependency, configuration, quality, security, release, and deployment reassessment.

A change to robotic control logic may require renewed functional, integration, operational, and physical-system verification.

The appropriate pathway depends on the governed reality rather than the

apparent size of the modification.

Lifecycle Re-entry ≠ Complete Lifecycle Repetition.

Historical Evidence Available ≠ Historical Evidence Still Applicable.

Change Authorized ≠ Lifecycle Obligations Satisfied.

Lifecycle Re-entry Identity

Every governed re-entry case must have a stable identity.

Relevant elements include:

Re-entry Case ID

Change Request ID

Software Product ID

Impact Assessment Reference

Coordination Reference

Creation Time

Responsible Governance Function

Current Re-entry Status

Evidence Reference

Re-entry Version

Material revisions must preserve their relationship to earlier re-entry decisions.

Lifecycle Re-entry Trigger

Lifecycle re-entry may be triggered by:

New Functional Requirement

Software Defect Correction

Security Vulnerability

Dependency Replacement

Configuration Modification

Build or Artifact Change

Quality Failure

Release Restriction

Deployment Failure

Operational Incident

Data Migration

Infrastructure Transition

Autonomous Agent Modification

Retirement-Related Change

The trigger must be associated with an identifiable software change and its assessed consequences.

Re-entry Necessity

LRECM must determine whether a proposed change materially affects an existing lifecycle governance conclusion.

Relevant questions include:

Does the change alter a governed software object?

Does it invalidate an existing requirement?

Does it introduce or modify a dependency?

Does it affect configuration or artifact identity?

Does it alter quality or security conditions?

Does it affect release or deployment readiness?

Does it change operational assumptions?

Does it introduce new verification obligations?

A negative re-entry determination must be supported by a documented assessment basis.

Lifecycle State Reference

LRECM must identify the relevant lifecycle condition of the affected software.

Potential states include:

Requirements Definition

Software Creation

Dependency Assessment

Configuration Establishment

Quality Verification

Security Verification

Release Preparation

Release Authorization

Deployment Preparation

Production Operation

Change Processing

Retirement Preparation

Retired

A software product may simultaneously have different components or versions occupying different lifecycle states.

Re-entry Scope

The re-entry scope identifies the lifecycle governance mechanisms requiring

renewed or additional activity.

Relevant elements include:

Affected Software Objects

Affected Parent Standards

Affected Core Modules

Required Assessments

Required Implementation Activities

Required Verification

Required Decisions

Applicable Environments

Required Evidence

Known Exclusions

The scope must remain traceable to the change impact assessment.

Re-entry Boundary

The re-entry boundary distinguishes required lifecycle activities from

unaffected governance mechanisms.

Relevant boundaries include:

Functional Boundary

Artifact Boundary

Dependency Boundary

Configuration Boundary

Security Boundary

Quality Boundary

Environment Boundary

Operational Boundary

Authority Boundary

Evidence Applicability Boundary

Exclusion of a lifecycle activity must not conceal a material unresolved consequence.

Lifecycle Impact Mapping

LRECM must map assessed change consequences to the relevant lifecycle governance mechanisms.

The mapping must preserve:

Change Impact Reference

Affected Software Object

Affected Governance Requirement

Relevant Parent Standard

Relevant Core Module

Required Activity

Evidence Requirement

Completion Condition

Dependency Reference

This mapping forms the basis of the lifecycle re-entry pathway.

Requirements Re-entry

Re-entry into SRS may be required when a change affects:

Software Purpose

Functional Requirements

Nonfunctional Requirements

Requirements Scope

Requirements Consistency

Requirements Traceability

Requirements Change Control

LRECM identifies the required requirements governance activities.

SRS governs their execution and acceptance.

Software Creation Re-entry

Re-entry into SCS may be required when a change affects:

Source Code

Software Artifacts

Development Method

Creator Attribution

Automated Software Creation

Implementation Traceability

Development Evidence

A software change may require new creation activities without requiring every original creation decision to be repeated.

Dependency Governance Re-entry

Re-entry into SDS may be required when a change affects:

Dependency Identity

Dependency Origin

Dependency Version

Dependency Suitability

Transitive Dependencies

Component Introduction

Dependency Compatibility

Dependency Continuity

A dependency update must not bypass suitability or compatibility governance merely because an automated tool recommends it.

Configuration Governance Re-entry

Re-entry into SCFS may be required when a change affects:

Configuration Baseline

Build Definition

Build Inputs

Artifact Identity

Software Version

Configuration Change Control

Build Reproducibility

Artifact Equivalence

A configuration change may require renewed verification even when source code remains unchanged.

Quality Governance Re-entry

Re-entry into SQS may be required when a change affects:

Quality Requirements

Quality Criteria

Functional Correctness

Regression Exposure

Test Coverage

Defect Status

Performance

Reliability

Compatibility

The required verification scope must reflect the actual consequences of the modification.

Security Governance Re-entry

Re-entry into SSS may be required when a change affects:

Security Requirements

Attack Surface

Authentication

Authorization

Software Exposure

Security Testing

Vulnerability Conditions

Security Remediation

Security Configuration

A security-motivated change may require verification of both the intended remediation and any new security consequences.

Release Governance Re-entry

Re-entry into SRLS may be required when a change affects:

Release Candidate Composition

Release Readiness

Release Verification Evidence

Release Authorization Conditions

Release Records

Release Compatibility

Release Withdrawal Conditions

An existing release authorization must not automatically be treated as applicable to a materially modified release candidate.

Deployment Governance Re-entry

Re-entry into SDPS may be required when a change affects:

Deployment Planning

Deployment Authorization

Production Version

Deployment State

Operational Monitoring

Rollback Feasibility

Recovery Requirements

Production Compatibility

A change may require deployment governance even when its implementation has already passed development and quality verification.

Retirement Governance Re-entry

Re-entry into SRTS may be required when a change affects:

Retirement Planning

End-of-Support Conditions

Migration Requirements

Residual Dependencies

Data Continuity

Retirement Closure

Replacement Compatibility

A retirement-related modification may require renewed lifecycle activity before the affected software can be legitimately withdrawn.

Lifecycle Control Interaction

SLCS provides cross-cutting governance for lifecycle state consistency,

transitions, exceptions, control transfer, and continuity.

LRECM must identify where a change affects:

Lifecycle State Consistency

Lifecycle Transition Conditions

Lifecycle Exceptions

Lifecycle Control Ownership

Lifecycle Continuity

Lifecycle Control Assessment

LRECM governs change-driven re-entry.

SLCS governs the integrity of the broader lifecycle control mechanisms.

Re-entry Granularity

Lifecycle re-entry may be required at different levels.

Potential levels include:

Individual Governance Requirement

Specific Core Module Activity

Core Module Reassessment

Parent Standard Reassessment

Multiple Parent Standards

Cross-Architecture Governance

The selected level must be justified by the assessed change consequences.

A change affecting one requirement does not automatically require repeating an entire Parent Standard.

Proportional Re-entry

LRECM must establish a proportional relationship between change impact and required lifecycle activity.

Relevant considerations include:

Change Significance

Affected Objects

Impact Propagation

Existing Evidence

Verification Coverage

Operational Criticality

Security Exposure

Uncertainty

Reversibility

Applicable Mandatory Requirements

Proportionality must not be used to bypass mandatory verification.

Historical Evidence Identification

LRECM must identify existing evidence potentially relevant to the modified software state.

Evidence may include:

Requirements Records

Development Records

Dependency Assessments

Configuration Baselines

Build Evidence

Quality Test Results

Security Assessments

Release Records

Deployment Records

Operational Observations

Historical evidence must remain associated with its original scope and conditions.

Evidence Applicability Assessment

Before historical evidence is reused, LRECM must evaluate whether its assumptions remain valid.

Relevant elements include:

Software Version

Artifact Identity

Dependency State

Configuration

Test Environment

Operational Conditions

Verification Scope

Evidence Age

Changed Requirements

Known Limitations

Evidence reuse must be supported by an identifiable applicability basis.

Evidence Reuse

Historical evidence may be reused when it remains materially applicable to the changed software state.

Potential classifications include:

Reusable Without Material Change

Reusable With Additional Verification

Partially Reusable

Not Reusable

Applicability Uncertain

The decision must preserve the supporting rationale and relevant evidence references.

Evidence Exists ≠ Evidence Remains Valid.

Reverification Requirements

Reverification is required where material changes invalidate or limit earlier verification conclusions.

Relevant elements include:

Affected Requirement

Changed Software Object

Previous Verification

Invalidated Assumption

Required New Verification

Verification Environment

Acceptance Condition

Evidence Reference

Reverification must target the affected governance conditions without unjustified duplication.

New Governance Obligations

A change may introduce obligations that did not exist in the previous software state.

Examples include:

New Security Requirement

New Dependency Assessment

New Compatibility Test

New Data Migration Verification

New Operational Monitoring Requirement

New Recovery Requirement

New Interface Validation

New Human Oversight Condition

LRECM must distinguish new obligations from repeated historical activities.

Re-entry Preconditions

Each required lifecycle activity may have entry conditions.

Potential conditions include:

Identifiable Change Request

Sufficient Impact Assessment

Required Coordination Established

Applicable Authority

Required Software Artifacts Available

Relevant Baseline Established

Necessary Resources Available

Prerequisite Governance Activity Completed

Material Uncertainty Addressed

An activity must not be classified as legitimately entered when mandatory prerequisites remain unsatisfied.

Re-entry Condition States

LRECM recognizes the following condition states:

Satisfied

Not Satisfied

Partially Satisfied

Unknown

Not Applicable

Exception

Each condition must preserve its evidence or applicable exception reference.

Unknown ≠ Satisfied.

Activity Scheduled ≠ Entry Conditions Established.

Re-entry Sequencing

Lifecycle activities may require a particular sequence.

Examples include:

Requirements Revision → Software Creation

Dependency Assessment → Configuration Revision

Configuration Revision → Quality Verification

Security Remediation → Security Reverification

Quality and Security Readiness → Release Reassessment

Release Authorization → Deployment Authorization

The sequence must reflect applicable governance dependencies rather than an assumption that every change must repeat the complete original lifecycle order.

Parallel Lifecycle Activities

Some lifecycle activities may proceed in parallel where their prerequisites and dependencies permit.

Relevant considerations include:

Independent Governance Scope

Shared Software Objects

Evidence Dependencies

Authority Conditions

Resource Constraints

Compatibility Requirements

Verification Interactions

Parallel processing must not compromise required sequencing or evidence validity.

Cross-Standard Dependency

A lifecycle re-entry pathway may contain dependencies between standards.

Relevant elements include:

Source Governance Activity

Dependent Governance Activity

Required Evidence

Prerequisite Condition

Completion Status

Failure Response

Reassessment Requirement

LRECM must preserve these dependencies within the Software Lifecycle Re-entry Profile.

Re-entry Decision

Every material re-entry determination must preserve:

Decision Reference

Change Request

Impact Assessment

Affected Governance Mechanisms

Required Activities

Excluded Activities

Evidence Reuse Decisions

Applicable Authority

Decision Conditions

Decision Time

A re-entry decision establishes the required governance pathway.

It does not independently authorize every technical action within that pathway.

Authority and Permission Boundary

LRECM must preserve the distinction between authority to direct a change into lifecycle governance and authority to execute specific activities.

Relevant elements include:

Decision Authority

Implementation Authority

Verification Responsibility

Release Authority

Deployment Authority

Agent Permissions

Delegation Conditions

Escalation Requirements

Lifecycle Re-entry Decision ≠ Universal Execution Permission.

Change Authorization Interaction

A software change may have an applicable change-level decision while still requiring specialized lifecycle decisions.

Examples include:

Change Accepted

Development Activity Permitted

Security Verification Required

Release Authorization Pending

Deployment Authorization Pending

These conditions must remain distinct.

An approved change must not bypass required release or deployment authorization.

Automated Lifecycle Re-entry

Automated systems may support lifecycle re-entry determination and execution.

Examples include:

Automated Test Selection

Dependency Assessment Triggers

Configuration Validation

Security Scanning

Build Reproduction

Release Readiness Checks

Deployment Preconditions

Automation must preserve:

Trigger Identity

Applicable Change

Selected Governance Activity

Selection Basis

Execution Authority

Evidence

Failure Conditions

Automated selection must remain subject to the actual governance requirements of the affected lifecycle mechanism.

AI-Assisted Re-entry Determination

AI systems may recommend which lifecycle activities should be revisited.

LRECM must preserve:

AI System Identity

Input Change Context

Impact Assessment Reference

Recommended Lifecycle Activities

Recommendation Basis

Known Limitations

Required Review

Applicable Authority

An AI recommendation must not be treated as an independently established governance decision without an appropriate validation and authority basis.

Autonomous Agent Re-entry Boundary

Autonomous agents may participate in lifecycle activities where legitimately authorized.

Relevant elements include:

Agent Identity

Assigned Objective

Permitted Lifecycle Activities

Tool Permissions

Affected Software Objects

Environment Restrictions

Decision Boundaries

Human Override

Escalation Conditions

Execution Evidence

Authority to perform one lifecycle activity does not automatically establish authority for subsequent release or deployment actions.

Emergency Lifecycle Re-entry

Urgent changes may require accelerated lifecycle processing.

Potential triggers include:

Critical Security Vulnerability

Production Service Failure

Severe Operational Incident

Critical Dependency Failure

Safety-Relevant Software Condition

Infrastructure Continuity Threat

Emergency processing may alter the sequence or timing of applicable activities where legitimately permitted.

It must not erase the underlying governance obligations.

Emergency Re-entry Conditions

Emergency re-entry must preserve:

Emergency Origin

Affected Software

Urgency Evidence

Required Activities

Applicable Authority

Permitted Accelerations

Deferred Verification Obligations

Operational Restrictions

Post-Implementation Verification

Closure Conditions

Deferred obligations must remain explicitly governed until resolved.

Emergency Acceleration ≠ Governance Exemption.

Lifecycle Re-entry Exception

An exception may be required when a normal lifecycle activity cannot be performed under the applicable conditions.

Relevant elements include:

Exception ID

Affected Governance Requirement

Reason

Applicable Authority

Alternative Control

Residual Risk

Duration or Validity Condition

Required Follow-up

Evidence

Exceptions must not be inferred from missing records or incomplete implementation.

Re-entry Exception Boundary

A lifecycle exception must remain limited to its legitimate scope.

It must not automatically authorize:

Unrelated Software Modifications

Unverified Dependency Changes

Unrestricted Production Access

Bypassing Unaffected Mandatory Requirements

Uncontrolled Agent Actions

Permanent Removal of Deferred Obligations

The exception must remain traceable to the relevant governance requirement and competent decision.

Re-entry Execution Traceability

LRECM must preserve the relationship between required lifecycle activities and their actual processing.

Relevant elements include:

Required Activity

Responsible Function

Applicable Standard

Applicable Core Module

Entry Condition

Execution Reference

Evidence

Completion Status

Outstanding Obligation

Technical execution records may be referenced without replacing the specialized governance records.

Re-entry Activity Status

Each required activity must have an identifiable status.

Potential states include:

Not Started

Entry Conditions Pending

Ready

In Progress

Verification Pending

Completed With Evidence

Failed

Blocked

Deferred Under Exception

Reassessment Required

Unknown

An activity marked complete without sufficient evidence must not automatically be treated as a satisfied governance obligation.

Re-entry Verification

LRECM must verify whether required lifecycle activities have produced the applicable evidence and conclusions.

Relevant elements include:

Required Activity

Completion Evidence

Applicable Acceptance Criteria

Responsible Governance Function

Evidence Validity

Unresolved Findings

Completion Decision

Remaining Dependencies

LRECM verifies the fulfillment of the re-entry pathway.

It does not replace the substantive verification performed under specialized standards.

Re-entry Completion Conditions

A lifecycle re-entry case may be considered complete only when its applicable completion conditions are established.

Relevant conditions include:

Required Activities Identified

Mandatory Preconditions Satisfied

Applicable Decisions Recorded

Required Evidence Available

Required Verification Completed

Material Conflicts Resolved

Outstanding Exceptions Identified

Residual Obligations Preserved

Handoff Conditions Established

Completion must be supported by evidence appropriate to the governed change.

Partial Re-entry Completion

Some required lifecycle activities may be completed while others remain unresolved.

LRECM must preserve:

Completed Activities

Pending Activities

Failed Activities

Deferred Activities

Outstanding Verification

Unresolved Dependencies

Applicable Restrictions

Required Follow-up

Partial completion must not be presented as full lifecycle compliance.

Re-entry Failure

Re-entry failure may occur when:

Required Activity Cannot Be Performed

Mandatory Preconditions Fail

Verification Fails

Required Authority Is Missing

Evidence Is Insufficient

Dependency Conditions Become Invalid

Operational Conditions Become Unacceptable

Required Governance Activity Is Bypassed

Failure must trigger appropriate reassessment, restriction, escalation, or transfer to another governed pathway.

Re-entry Deviation

A deviation occurs when actual lifecycle processing differs materially from the established re-entry pathway.

Examples include:

Unplanned Activity Omission

Unapproved Sequence Change

Different Software Artifact

Changed Dependency

Unexpected Environment

Unverified Evidence Reuse

Unauthorized Automated Execution

Incomplete Verification

Material deviations must be assessed before the pathway can be treated as completed.

Re-entry Reassessment

Reassessment may be required when:

Change Scope Expands

Impact Assessment Changes

Coordination Conditions Change

New Dependencies Are Identified

Historical Evidence Becomes Inapplicable

Verification Fails

Implementation Deviates

Operational Conditions Change

New Security Findings Emerge

An Agent Performs an Unplanned Action

Reassessment must preserve the earlier pathway and identify which conclusions remain valid.

Lifecycle Re-entry Continuity

LRECM must preserve continuity across:

Change Revisions

Lifecycle Reassessment

Actor Changes

Responsibility Transfers

Interrupted Implementation

Deferred Activities

Emergency Processing

Automated Execution

Repeated Verification

Long-Term Software Maintenance

The history of required activities, decisions, evidence, and unresolved

obligations must remain reconstructable.

Residual Lifecycle Obligations

A lifecycle re-entry case may retain obligations after particular activities have been completed.

Examples include:

Deferred Security Verification

Outstanding Compatibility Test

Post-Deployment Monitoring

Data Migration Verification

Recovery Validation

External Integration Confirmation

Physical-System Validation

Exception Expiration Review

Residual obligations must remain associated with responsible functions and applicable resolution conditions.

Re-entry Governance Classification

LRECM recognizes the following governance classifications:

Re-entry Required

Re-entry Not Required With Justification

Re-entry Scope Under Assessment

Re-entry Pathway Established

Entry Conditions Pending

Ready for Lifecycle Processing

Lifecycle Processing Active

Evidence Reuse Approved

Reverification Required

Partially Completed

Completed With Evidence

Blocked

Failed

Exception Active

Residual Obligations Outstanding

Reassessment Required

Transferred to Change Closure

Unknown

Conflicted

Classification must remain associated with the relevant re-entry case, scope, version, and evidence.

Governance Requirements

A conforming LRECM implementation must satisfy the following requirements. Re-entry Identity and Determination

- Establish a stable lifecycle re-entry case identity
- Associate the case with an identifiable software change
- Preserve relevant impact assessment references
- Preserve relevant coordination references
- Identify the affected software objects
- Establish the current lifecycle state
- Determine whether lifecycle re-entry is required
- Preserve the basis for the determination

Scope and Governance Mapping

- Identify materially affected Parent Standards
- Identify applicable Core Modules
- Establish required lifecycle activities
- Define the re-entry scope
- Preserve justified exclusions
- Identify cross-standard dependencies
- Establish required activity sequencing
- Identify permitted parallel activities

Evidence and Verification

• Identify relevant historical evidence

• Evaluate historical evidence applicability

• Preserve evidence reuse decisions

• Identify required reverification

• Identify new governance obligations

• Establish required evidence for each activity

• Preserve specialized verification references

• Prevent unsupported evidence reuse

Authority and Conditions

• Identify applicable decision authority

• Preserve execution authority boundaries

• Establish entry conditions

• Identify prerequisite activities

• Preserve applicable restrictions

• Govern emergency re-entry conditions

• Preserve exception authority and scope

• Prevent re-entry decisions from substituting for specialized authorization.

Execution and Continuity

• Preserve lifecycle activity status

• Maintain execution traceability

• Identify material deviations

• Establish reassessment triggers

• Preserve material pathway revisions

• Verify required activity completion

• Identify partial completion

• Preserve unresolved obligations

• Maintain historical lifecycle continuity

• Establish evidence-supported re-entry classification

• Provide structured references to CCLM

• Prevent incomplete lifecycle processing from being presented as fully satisfied.

Governance Logic

Software Change → Impact Assessment → Coordination References → Lifecycle

Re-entry Trigger → Current Lifecycle State → Affected Governance Mechanisms → Re-entry Necessity → Re-entry Scope → Historical Evidence → Evidence Applicability → Required Activities → Entry Conditions → Authority References → Lifecycle Sequence → Governed Processing → Verification Evidence → Completion Assessment → Residual Obligations → Software Lifecycle Re-entry Profile

Where historical evidence remains applicable:

Existing Evidence → Applicability Assessment → Reuse Decision → Preserved

Evidence Reference

Where evidence is invalidated:

Material Change → Invalidated Assumption → Reverification Requirement →

Specialized Governance Activity

Where multiple standards are affected:

Change Impact → Affected Standards → Dependency Mapping → Coordinated

Re-entry Pathway

Where emergency processing is required:

Emergency Condition → Applicable Authority → Accelerated Pathway →

Preserved Mandatory Obligations → Deferred Verification Tracking

Where an autonomous agent participates:

Agent Objective → Applicable Authority → Permitted Lifecycle Activity →

Tool-Path Boundary → Execution Evidence → Verification

Where implementation deviates:

Established Pathway → Material Deviation → Reassessment → Revised Pathway

/ Restriction / Escalation

LRECM must not equate a completed automation pipeline with satisfied lifecycle governance unless the required activities, evidence, and decisions have been established.

Governance Outputs

The principal output is:

Software Lifecycle Re-entry Profile

Supporting outputs may include:

Lifecycle Re-entry Case Identity

Change-to-Lifecycle Mapping

Affected Parent Standard Register

Affected Core Module Register

Re-entry Necessity Assessment

Re-entry Scope Record

Historical Evidence Register

Evidence Applicability Assessment

Evidence Reuse Decision Record

Reverification Requirement Register

Lifecycle Activity Register

Entry Condition Register

Cross-Standard Dependency Map

Lifecycle Sequence Profile

Authority Boundary Reference

Emergency Re-entry Record

Lifecycle Exception Register

Execution Traceability Record

Lifecycle Verification Record

Re-entry Reassessment Record

Residual Obligation Register

Re-entry Completion Assessment

Change Closure Handoff Record

Minimum Implementation Framework

A minimum LRECM implementation must establish eight governance layers. Layer 1 — Change and Lifecycle Identity

Identify the software change, current lifecycle state, relevant impact

assessment, and coordination references. Layer 2 — Governance Mapping

Determine which Parent Standards, Core Modules, and governance requirements

are materially affected. Layer 3 — Evidence Applicability

Identify historical evidence, assess its continued validity, and establish

reuse or reverification requirements. Layer 4 — Re-entry Pathway

Define required activities, applicable exclusions, sequencing,

dependencies, and completion conditions. Layer 5 — Authority and Preconditions

Identify responsible functions, decision boundaries, execution permissions,

mandatory prerequisites, and applicable exceptions. Layer 6 — Governed Processing

Preserve the status and evidence of required lifecycle activities without

replacing specialized governance mechanisms. Layer 7 — Verification and Continuity

Assess completion, identify deviations, preserve reassessment triggers, and

maintain unresolved obligations. Layer 8 — Governed Output and Handoff

Establish the Software Lifecycle Re-entry Profile and provide

evidence-supported references to CCLM.

The implementation may be manual, workflow-supported, automated, or AI-assisted.

The software used to implement LRECM must adapt to its governance requirements rather than redefine them.

Cross-Module Boundary

CRMM → LRECM

CRMM establishes the original change request identity, purpose, preliminary

scope, and evidence.

LRECM preserves these references while determining the applicable lifecycle

pathway.

CIAM → LRECM

CIAM identifies direct and indirect consequences, uncertainty, verification

implications, and affected lifecycle mechanisms.

LRECM determines which governance activities must be performed or revisited.

CHCM → LRECM

CHCM establishes coordination dependencies, actor relationships,

compatibility conditions, and sequencing constraints.

LRECM incorporates applicable coordination conditions into the re-entry

pathway.

LRECM → CCLM

LRECM establishes the required lifecycle obligations and their completion

status.

CCLM evaluates the resulting change outcome, outstanding obligations, and

legitimate closure disposition.

Lifecycle Processing Completed ≠ Change Outcome Verified.

Cross-Architecture Boundary

LRECM directs software changes into applicable SLGA™ governance mechanisms

without replacing their specialized requirements.

SRS — Software Requirements Standard

Requirements definition, scope, change control, and traceability.

SCS — Software Creation Standard

Software creation, artifact generation, and development traceability.

SDS — Software Dependency Standard

Dependency suitability, introduction, compatibility, and continuity.

SCFS — Software Configuration Standard

Configuration baselines, builds, artifact identification, and

reproducibility.

SQS — Software Quality Standard

Quality requirements, testing, defect governance, and quality condition

assessment.

SSS — Software Security Standard

Security requirements, exposure, testing, vulnerabilities, and remediation.

SRLS — Software Release Standard

Release candidates, readiness, authorization, records, and withdrawal.

SDPS — Software Deployment Standard

Deployment planning, authorization, production versions, monitoring,

rollback, and recovery.

SRTS — Software Retirement Standard

Retirement planning, end-of-support, migration, residual dependencies, and

closure.

SLCS — Software Lifecycle Control Standard

Lifecycle state consistency, transitions, exceptions, control transfer, and

continuity.

Applicable OOF® authority, accountability, autonomous systems, and

operational reality governance references remain independently applicable where the change affects those governed domains.

Practical Applications

Enterprise Software Maintenance

Directing changes into only those lifecycle governance activities

materially affected by the modification.

Continuous Integration and Delivery

Determining which verification and governance stages must be repeated after changes to software artifacts, dependencies, or configurations.

Security Remediation

Establishing appropriate dependency, configuration, quality, security,

release, and deployment re-entry after vulnerability corrections.

AI-Assisted Software Development

Preventing AI-generated patches from bypassing necessary lifecycle

governance while avoiding unnecessary repetition of unaffected activities.

Autonomous Software Operations

Constraining agents to legitimately assigned lifecycle activities and

preserving the distinction between automated execution and governance completion.

Critical Infrastructure

Establishing traceable lifecycle re-entry for changes affecting service

continuity, security, operational restrictions, or recovery capability.

Robotics and Embedded Systems

Determining when firmware or control logic modifications require renewed integration, operational, or physical-system verification.

Use Case 1 — A Security Dependency Update Requires Selective Lifecycle

Re-entry

A software team identifies a security vulnerability in a third-party

dependency.

A corrected dependency version is available.

The proposed update does not alter the application's stated functional requirements.

However, the new dependency version changes internal interfaces and

introduces different runtime behavior.

LRECM evaluates:

Change Request → Impact Assessment → Dependency Change → Configuration

Consequences → Quality Verification → Security Verification → Release and Deployment Requirements

The assessment establishes:

• Existing functional requirements remain applicable

• Dependency suitability must be reassessed

• Configuration and build compatibility must be verified

• Relevant regression testing is required

• Security remediation effectiveness must be verified

• Release and deployment governance must be revisited where the updated

• Unaffected requirements-definition activities do not need to be

artifact is introduced. repeated without justification.

The appropriate classification is:

Selective Lifecycle Re-entry Required — Multiple Affected Governance

Mechanisms.

A dependency update does not require repeating every lifecycle activity, but it must satisfy every lifecycle obligation materially affected by the update.

Use Case 2 — An AI-Generated Patch Passes Tests but Cannot Bypass

Deployment Governance

An autonomous software maintenance agent is authorized to propose and implement limited corrections within a development environment.

The agent identifies a defect, generates a patch, and executes the

permitted automated tests.

The tests pass.

The agent then attempts to promote the modified artifact directly into

production.

LRECM evaluates:

Agent-Generated Change → Creation Evidence → Configuration and Artifact

Identity → Quality Verification → Security Implications → Release Requirements → Deployment Authorization

The assessment establishes:

• The agent's development activities are within its assigned scope

• Relevant test evidence is available

• The modified artifact requires applicable configuration and release

• Production deployment requires a distinct authorization basis

• Passing development tests does not establish permission to execute

• The agent's tool permissions must remain restricted to its legitimate

governance. production deployment. authority.

The appropriate classification is:

Lifecycle Re-entry Partially Satisfied — Release and Deployment

Obligations Outstanding.

Automated Test Success ≠ Authorization to Enter Production.

Critical Distinctions

Lifecycle Re-entry ≠ Complete Lifecycle Repetition.

Change Authorized ≠ Lifecycle Obligations Satisfied.

Historical Evidence Available ≠ Historical Evidence Still Applicable.

Evidence Exists ≠ Evidence Remains Valid.

Evidence Reuse ≠ Verification Bypass.

Previously Completed Activity ≠ Currently Satisfied Requirement.

Limited Modification ≠ Limited Lifecycle Impact.

Affected Standard Identified ≠ Required Activity Completed.

Re-entry Decision ≠ Universal Execution Permission.

Activity Scheduled ≠ Entry Conditions Established.

Unknown ≠ Satisfied.

Emergency Acceleration ≠ Governance Exemption.

Automated Pipeline Completion ≠ Lifecycle Governance Completion.

Agent Task Authority ≠ Authority for Every Lifecycle Action.

Partial Re-entry Completion ≠ Full Lifecycle Compliance.

Lifecycle Processing Completed ≠ Change Outcome Verified.

Canonical Boundary

LRECM governs the determination, structuring, evidence applicability,

sequencing, execution traceability, and verification of software lifecycle governance activities required because of a proposed or implemented software change.

It establishes the relationship between assessed change consequences and

the specific lifecycle mechanisms that must be performed, repeated, extended, or verified.

It preserves justified evidence reuse, required reverification, authority

boundaries, exceptions, material deviations, and unresolved lifecycle obligations.

LRECM does not replace specialized lifecycle standards, independently authorize implementation, or establish final change closure.

Its principal governance output is the:

Software Lifecycle Re-entry Profile

This output provides the evidence-supported record of which lifecycle

governance mechanisms were required, why they were required, how their obligations were addressed, and which conditions remain unresolved.

Canonical Closing Statement

Software lifecycle governance loses its meaning when changes can invalidate earlier conclusions without triggering the activities necessary to establish their continued validity.

A security patch may require renewed dependency and quality assessment.

A configuration change may invalidate previously verified runtime conditions.

A new interface may require integration testing even when its individual components function correctly.

A deployment correction may require renewed release and production authorization.

An AI-generated modification may satisfy development tests while remaining outside its legitimate operational authority.

And a minor change may require extensive governance because of the consequences it introduces, while a larger change may leave some historical evidence fully applicable.

LRECM establishes the methodological mechanism required to govern these

differences.

It does not require every software change to restart the entire lifecycle.

It does not permit necessary lifecycle obligations to disappear merely because a modification is urgent, automated, or technically simple.

Instead, it determines which governance mechanisms are materially affected, which historical evidence remains applicable, which activities must be repeated or extended, and which conditions must be verified before the resulting software state can be considered adequately governed.

It preserves the distinction between a completed technical action and a

satisfied governance requirement.

It maintains continuity across lifecycle revisions, exceptions,

reassessments, and automated execution.

And it ensures that unresolved obligations remain visible until they are legitimately resolved or transferred.

The purpose of lifecycle re-entry is not to repeat the past. It is to determine which conclusions from the past remain valid after software reality has changed—and to re-establish those that no longer do.

Lifecycle Re-entry Control Module (LRECM) transforms software change

processing into a proportional, evidence-supported mechanism for preserving lifecycle governance integrity as software evolves.